

Roteiro Prático: Dominando o Git do Zero

Introdução: O Mapa da Mina (As Três Áreas do Git)

O **Git** funciona como um sistema de estados lógicos. Um arquivo não é simplesmente salvo; ele atravessa zonas de maturidade antes de se tornar parte do **histórico imutável**.

Área	O que acontece nela	Comando Principal de Transição
Working Directory	Sua área de "rascunho". Arquivos editados aqui são considerados <i>untracked</i> ou <i>modified</i> .	git add <arquivo>
Staging Area (Index)	Onde você prepara o próximo <i>snapshot</i> . É o filtro de qualidade para o que será auditado.	git commit -m "mensagem"
Repository (.git)	Onde reside o Audit Trail . Cada commit gera um Hash SHA-1 único e permanente.	(Registro Permanente)

Entender essas áreas evita o "medo contínuo" de quebrar o código. Antes de qualquer ação, precisamos configurar sua identidade profissional para garantir a rastreabilidade em caso de incidentes.

Atividade 1: Configuração e o Incidente de R\$ 50k

O Cenário de Crise: Em uma startup sem governança, um dev ignorou a "Síndrome do Arquivo Final" e subiu logs de *debug* pesados e chaves de API expostas. Sem um **.gitignore**, o repositório inflou e as credenciais vazaram. Resultado: uma fatura de **\$50.000,00** na **AWS** e um incidente crítico de segurança.

Passo a Passo Técnico:

1. **Identidade Profissional:** Configure os metadados de autoria. Note que este e-mail aparecerá no histórico de commits para auditoria, mas não é o seu login de autenticação.

```
git config --global user.name "Seu Nome Profissional"
git config --global user.email "seu@email.com"
```

1. **Inicialização do Repositório:** Crie o ambiente resiliente.

```
mkdir sistema-critico && cd sistema-critico
git init
```

1. **Higiene de Código (.gitignore):** Impeça que arquivos temporários (`x.txt`) ou lixo de sistema poluam o **Repository**.

```
echo "x.txt" >> .gitignore
echo "*.log" >> .gitignore
echo ".env" >> .gitignore
```

Conexão: Com a casa protegida, o próximo passo é estabelecer o fluxo de **Code Review** local antes de qualquer entrega.

Atividade 2: O Fluxo Profissional de Code Review

Trabalhar em alta performance exige que você seja o seu primeiro revisor. O objetivo é garantir que nenhum "lixo" chegue ao **stage**.

Auditoria de Estado: Verifique o que está pendente.

```
git status
```

Análise de Diferenças (Working Directory): Veja o que você alterou antes de promover o arquivo.

```
git diff
```

Promovendo ao Stage e Revisão Final:

```
git add .  
git diff --staged # Fundamental para revisar o que está no Index
```

Gravação do Snapshot: Use mensagens semânticas.

```
git commit -m "feat: setup inicial de segurança e governança"
```

Consulta ao Log Detalhado: Analise o **patch** da última mudança (use 'q' para sair).

```
git log -p
```

Conexão: Commits diretos na **branch main** são um antipadrão. Demandas reais, como as do **Jira**, exigem isolamento via **branches**.

Atividade 3: Feature Branches e o Hotfix de Black Friday

Cenário de Alta Disponibilidade: Você está na **branch** `feat/PROJ-101-pagamento-pix`. No meio do código, surge um alerta: o botão de compra quebrou em plena Black Friday. Você precisa trocar de contexto sem perder o que fez.

Passo a Passo Técnico:

1. **Criação da Feature:** Use o comando moderno `switch`.

```
git switch -c feat/PROJ-101-pagamento-pix
```

2. **Alerta de Crise (Hotfix):** Volte para a linha estável e corrija o bug imediatamente.

```
git switch main  
git switch -c hotfix/botao-comprar  
echo "fix: ajuste no trigger do botao" > botao.js  
git add botao.js  
git commit -m "fix(ui): corrige trigger de clique no checkout"
```

3. **Tag de Release:** Marque o deploy com uma **tag anotada** para o time de infra.

```
git tag -a v1.0.0 -m "Hotfix emergencial Black Friday - Recuperacao de SLA"
```

Conexão: Na pressa do **hotfix**, erros de commit são comuns. Para evitar poluir o histórico com "fix do fix", usaremos a técnica do reparo perfeito.

Atividade 4: O Acidente e o Reparo Perfeito

4.1 O Cenário do Erro

Você subiu o arquivo `app.js` de autenticação, mas esqueceu o template de variáveis de ambiente (`.env.example`).

```
Modifique o app.js
git add app.js
git commit -m "add login"
```

4.2 A Descoberta e Gravidade

Esse commit incompleto causará um **Build Break** no pipeline de CI/CD. Seus colegas não conseguirão subir o ambiente local sem o `.env.example`.

4.3 O Reparo com Amend

Não gere um novo commit. Vamos reescrever a história local para anexar o arquivo esquecido e corrigir a mensagem para o padrão de **Commits Semânticos**.

```
Crie/modifique o arquivo .env.example
git add .env.example
git commit --amend -m "feat(auth): implementa autenticação e template de env"
```

4.4 Comprovação Técnica

Execute o comando abaixo para ver o **patch** do último commit e notar que ele agora contém ambos os arquivos sob o mesmo **Hash SHA-1** atualizado. Pode usar a estrutura de visualização do VS Code também.

```
git log -p -1
```

4.5 !!! ALERTA DE SEGURANÇA E ARQUITETURA !!!

O Aviso do Especialista: O `--amend` é uma operação destrutiva no histórico. Ele apaga o commit anterior e cria um novo com um novo **Hash SHA-1**.

- **REGRA DE OURO:** O **amend** é apenas para uso **LOCAL**.
- **O Desastre:** Nunca use **amend** em commits que já sofreram **push**. Se você fizer isso e tentar um **push --force**, você destruirá o trabalho da equipe, causando conflitos massivos e sobrescrevendo o código de outros desenvolvedores.

Atividade 5: Restores Modernos e Segurança de Dados

Se você cometeu um erro de **LGPD** (adicionou dados sensíveis ao stage) ou quebrou o código local, use os comandos modernos que separam a responsabilidade de arquivos e ramos. Ou adicionou alguma senha ou chave de API :-).

Comando Antigo	Comando Moderno (Recomendado)	Ação Executada	Gravidade
git reset HEAD <file>	git restore --staged <file>	Tira o arquivo da Staging Area (rebaixa para o Working Dir).	Baixa
git checkout -- <file>	git restore <file>	Descarta mudanças locais. O arquivo volta ao estado do último commit.	ALTA (Irreversível)

Conexão: Com o histórico limpo e auditado, é hora de integrar seu código ao ecossistema global.

Atividade 6: Colaboração Remota e Portfólio Profissional

A entrega final é o que define sua senioridade. O **GitHub** é sua vitrine técnica.

1. **Sincronização e Publicação:** Antes de enviar, use o **git fetch** para atualizar as referências das **branches** remotas dos seus colegas.

```
git fetch
git branch -r # Lista branches no servidor
git push -u origin feat/PROJ-101-pagamento-pix
(se você baixou o projeto pelo git clone, é só usar git push).
```

1. **Autenticação Profissional:**

- **GitHub CLI (gh auth login):** A escolha ágil para automação.
- **Chaves SSH (RSA/Ed25519):** O padrão ouro para conexões seguras sem senhas.
- **PAT (Personal Access Token):** Para autenticação granular via HTTPS.

2. **O README.md como Documentação de Engenharia:** Não trate o **README** apenas como texto. Ele é sua especificação técnica. Inclua:

- **Tech Stack:** Ferramentas e versões.
- **Instruções de Build:** Como rodar o projeto do zero (crucial para SREs).
- **Utilidade:** Descreva o problema que o código resolve.

Conclusão Final: O **Git** não é burocracia; é a infraestrutura que garante sua liberdade criativa. Ao dominar a imutabilidade do histórico e a gestão de branches, você deixa de ser um codificador para se tornar um arquiteto de soluções confiáveis. **Bom código!**